

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: G.Sravva

Roll No.: A24126552257

Date: 19-09-2026

TASK 4 — DOCUMENT OBJECT MODEL (DOM) EXPLANATION & DYNAMIC MANIPULATION (CO-1)

1. TITLE

Comprehensive Analysis of the Document Object Model (DOM) and Demonstration of Dynamic Content, Style, and Attribute Manipulation Using JavaScript

2. AIM

To explain the architecture, hierarchy, and mechanics of the Document Object Model (DOM), and construct an interactive web application demonstrating how JavaScript dynamically accesses and modifies the content, inline styles, CSS classes, and attributes of HTML elements in real time without refreshing the page.

3. OBJECTIVE

The objectives of this task are:

- To understand the hierarchical tree structure of the Document Object Model (DOM) generated by the browser.
- To contrast nodes and elements (Node vs Element) in the DOM hierarchy.
- To demonstrate dynamic content modification using `textContent`, `innerText`, and `innerHTML`.
- To demonstrate dynamic style manipulation using both direct CSS properties (`element.style.<property>`) and class list manipulation (`classList.add`, `classList.remove`, `classList.toggle`).
- To demonstrate dynamic attribute manipulation using `setAttribute()`, `getAttribute()`, `removeAttribute()`, and direct DOM properties (`src`, `href`, `disabled`).
- To build an interactive sandbox with real-time feedback and a live code inspector showing executed JavaScript commands.

4. THEORY

4.1 What is the Document Object Model (DOM)?

The Document Object Model (DOM) is a language-agnostic programming interface defined by the World Wide Web Consortium (W3C) that represents an HTML or XML document as a structured tree of objects. When an HTML document is fetched, the browser's rendering engine parses the markup and constructs an in-memory object hierarchy.

Every element, attribute, and snippet of text in the HTML markup corresponds to a node in this tree. JavaScript can access and manipulate these nodes programmatically to alter page structure, style, and content dynamically without initiating HTTP network requests.

4.2 Node Hierarchy and Types

Everything in the DOM is a Node. The primary node types are:

1. **Document Node (nodeType === 9):** The entry point to the DOM tree (`window.document`).
2. **Element Node (nodeType === 1):** Represents an HTML tag (e.g., `<div>`, `<p>`, `<a>`).
3. **Attribute Node (nodeType === 2):** Represents an attribute of an element (e.g., `class="btn"`, `id="card"`).
4. **Text Node (nodeType === 3):** Contains the textual content inside an element. Text nodes cannot contain child nodes.
5. **Comment Node (nodeType === 8):** Represents source comments within the markup (`<!-- comment -->`).

4.3 Dynamic Content Manipulation

JavaScript provides three primary properties for altering element content:

- **textContent:** Accesses or sets raw textual content. It ignores HTML markup and does not trigger HTML parsing. It is secure against Cross-Site Scripting (XSS).
- **innerText:** Reflects the rendered text visible to the user, accounting for CSS styles like `display: none` and `visibility: hidden`. It causes browser reflow and is slightly slower than `textContent`.
- **innerHTML:** Parses a string as HTML markup and injects child DOM nodes. While powerful for inserting markup, it can introduce XSS vulnerabilities if used with unvalidated user input.

4.4 Dynamic Style Manipulation

Styles can be modified through two mechanisms:

1. **Inline Style Modification (`element.style.<property>`):**
 - Applies styles directly to the element's style attribute.
 - Property names are written in camelCase (e.g., `backgroundColor`, `fontSize`, `borderRadius`).
2. **Class-Based Styling (`element.classList`):**
 - Modifies the element's CSS class names, maintaining separation of concerns between structure and presentation.
 - Methods: `classList.add()`, `classList.remove()`, `classList.toggle()`, `classList.contains()`, `classList.replace()`.

4.5 Dynamic Attribute Manipulation

HTML attributes configure element behavior and appearance. JavaScript can manipulate them through:

- `element.setAttribute(name, value)`: Adds or updates an attribute.
- `element.getAttribute(name)`: Retrieves the current string value of an attribute.
- `element.removeAttribute(name)`: Removes an attribute completely.
- `element.hasAttribute(name)`: Returns a boolean indicating whether the attribute exists.
- **Direct Property Access**: Built-in attributes can often be read and written directly as object properties (e.g., `img.src = 'photo.jpg'`, `button.disabled = true`, `a.href = 'https://...'`).

5. ALGORITHM / PROCEDURE

1. Create `index.html` with an interactive sandbox divided into:
 - **Target Demonstration Element**: A sample card containing a heading, description text, image, action button, and external link.
 - **Interactive Control Panels**: Three dedicated panels for Content, Style, and Attribute manipulation.
 - **Live JavaScript Code Inspector**: A code box showing the exact JavaScript statement executed on each button click.
2. Link the stylesheet `style.css` with layout styling, control button styles, and predefined utility classes (`.dark-theme`, `.highlight-border`, `.badge-accent`).
3. Create `script.js` to implement the DOM manipulation logic.
4. **Implement Content Manipulation**:
 - Attach click handlers to update heading text using `.textContent`.
 - Inject HTML badges and formatting using `.innerHTML`.
 - Dynamically append new list items using `document.createElement()` and `appendChild()`.
5. **Implement Style Manipulation**:
 - Modify inline styles directly: `element.style.backgroundColor`, `element.style.color`, `element.style.fontSize`.
 - Toggle pre-styled theme classes using `element.classList.toggle('dark-theme')`.
 - Toggle accent borders using `element.classList.toggle('highlight-border')`.
6. **Implement Attribute Manipulation**:
 - Swap image source and alternate text using `setAttribute('src', ...)` and `setAttribute('alt', ...)`.
 - Modify link destinations using `element.setAttribute('href', ...)`.
 - Toggle boolean attributes by modifying `button.disabled` and updating visual styles.
 - Set tooltip attributes using `element.setAttribute('title', ...)`.
7. Update the code inspector panel after each interaction to display the executed JavaScript command.
8. Test all interactive controls in a web browser to verify dynamic DOM updates.

6. PROGRAM

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Manipulation Demonstration</title>
  <link rel="stylesheet" href="style.css">
```

```

</head>
<body>
  <div class="app-container">
    <header class="app-header">
      <h1>Document Object Model (DOM) Manipulation</h1>
      <p class="subtitle">Demonstrating Dynamic Content, Style, and Attribute Updates
via JavaScript</p>
    </header>
    <div class="main-content">
      <!-- TARGET DEMONSTRATION CARD -->
      <section class="display-section">
        <h2>Target HTML Element</h2>
        <div id="targetCard" class="demo-card" title="Default Element Tooltip">
          <div class="card-badge" id="cardBadge">Status: Normal</div>
          
          <h3 id="cardTitle">Default Component Title</h3>
          <p id="cardDescription">
            This is the original paragraph content. JavaScript can dynamically alter its
text, inline styles, classes, and attributes.
          </p>
          <ul id="featureList" class="feature-list">
            <li>Standard DOM Node Item 1</li>
            <li>Standard DOM Node Item 2</li>
          </ul>
          <div class="card-actions">
            <button id="targetBtn" class="btn btn-primary">Target Action
Button</button>
            <a id="targetLink" href="https://developer.mozilla.org" target="_blank"
class="card-link">Visit MDN Web Docs & rarr;</a>
          </div>
        </div>
        <!-- Live JavaScript Inspector -->
        <div class="inspector-card">
          <h3>Executed JavaScript Command</h3>
          <pre><code id="codeOutput">// Click any control button below to see the
executed DOM command</code></pre>
        </div>
      </section>
      <!-- CONTROLS DASHBOARD -->
      <aside class="controls-section">
        <!-- PANEL 1: CONTENT MANIPULATION -->
        <div class="control-panel">
          <h3>1. Content Manipulation</h3>
          <p class="panel-desc">Modify innerHTML, textContent, or append new child
nodes.</p>
          <div class="btn-grid">
            <button id="btnTextContent" class="ctrl-btn">Change textContent</button>
            <button id="btnInnerHTML" class="ctrl-btn">Inject HTML Badge</button>
            <button id="btnAddNode" class="ctrl-btn">Append New Node</button>

```

```

        <button id="btnResetContent" class="ctrl-btn ctrl-btn-reset">Reset
Content</button>
    </div>
</div>
<!-- PANEL 2: STYLE MANIPULATION -->
<div class="control-panel">
    <h3>2. Style Manipulation</h3>
    <p class="panel-desc">Change element.style properties or toggle CSS
classes.</p>
    <div class="btn-grid">
        <button id="btnChangeColor" class="ctrl-btn">Change Text Color</button>
        <button id="btnToggleTheme" class="ctrl-btn">Toggle Dark Theme</button>
        <button id="btnToggleBorder" class="ctrl-btn">Toggle Highlight
Border</button>
        <button id="btnIncreaseFont" class="ctrl-btn">Increase Font Size</button>
    </div>
</div>
<!-- PANEL 3: ATTRIBUTE MANIPULATION -->
<div class="control-panel">
    <h3>3. Attribute Manipulation</h3>
    <p class="panel-desc">Set or remove attributes (src, href, title, disabled).</p>
    <div class="btn-grid">
        <button id="btnSwapImage" class="ctrl-btn">Swap Image (src)</button>
        <button id="btnToggleDisabled" class="ctrl-btn">Toggle Button
disabled</button>
        <button id="btnChangeLink" class="ctrl-btn">Change Link (href)</button>
        <button id="btnSetTooltip" class="ctrl-btn">Update Tooltip (title)</button>
    </div>
</div>
</aside>
</div>
</div>
<script src="script.js"></script>
</body>
</html>

```

style.css

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
body {
    background: #f8fafc;
    color: #1e293b;
    padding: 30px 20px;
}
.app-container {

```

```
    max-width: 1100px;
    margin: 0 auto;
}
.app-header {
    text-align: center;
    margin-bottom: 30px;
}
.app-header h1 {
    font-size: 28px;
    color: #0f172a;
    margin-bottom: 6px;
}
.subtitle {
    color: #64748b;
    font-size: 15px;
}
.main-content {
    display: grid;
    grid-template-columns: 1.1fr 1fr;
    gap: 28px;
}
@media (max-width: 900px) {
    .main-content {
        grid-template-columns: 1fr;
    }
}
.display-section h2, .controls-section h2 {
    font-size: 18px;
    margin-bottom: 14px;
    color: #334155;
}
.demo-card {
    background: #ffffff;
    border-radius: 12px;
    padding: 24px;
    border: 2px solid #e2e8f0;
    box-shadow: 0 4px 14px rgba(0, 0, 0, 0.05);
    transition: all 0.3s ease;
    position: relative;
    margin-bottom: 20px;
}
.card-badge {
    display: inline-block;
    background: #e2e8f0;
    color: #475569;
    padding: 4px 10px;
    border-radius: 6px;
    font-size: 12px;
    font-weight: 600;
    margin-bottom: 14px;
}
```

```
}
.card-img {
  width: 100%;
  height: 160px;
  object-fit: cover;
  border-radius: 8px;
  margin-bottom: 14px;
  border: 1px solid #cbd5e1;
}
.demo-card h3 {
  font-size: 20px;
  color: #0f172a;
  margin-bottom: 10px;
}
.demo-card p {
  font-size: 14px;
  line-height: 1.6;
  color: #475569;
  margin-bottom: 14px;
}
.feature-list {
  margin-left: 20px;
  margin-bottom: 18px;
  font-size: 14px;
  color: #475569;
}
.card-actions {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.btn {
  padding: 8px 16px;
  border-radius: 6px;
  font-size: 13px;
  font-weight: 600;
  cursor: pointer;
  border: none;
  transition: 0.2s;
}
.btn-primary {
  background: #2563eb;
  color: #ffffff;
}
.btn-primary:hover {
  background: #1d4ed8;
}
.btn-primary:disabled {
  background: #94a3b8;
  cursor: not-allowed;
}
```

```
}
.card-link {
  color: #2563eb;
  text-decoration: none;
  font-size: 13px;
  font-weight: 600;
}
.card-link:hover {
  text-decoration: underline;
}
/* Dynamic Classes Toggled via JS */
.demo-card.dark-theme {
  background: #0f172a;
  color: #f8faff;
  border-color: #334155;
}
.demo-card.dark-theme h3 {
  color: #f8faff;
}
.demo-card.dark-theme p, .demo-card.dark-theme .feature-list {
  color: #cbd5e1;
}
.demo-card.highlight-border {
  border-color: #3b82f6;
  box-shadow: 0 0 16px rgba(59, 130, 246, 0.35);
}
.inspector-card {
  background: #0f172a;
  color: #f8faff;
  border-radius: 10px;
  padding: 16px 20px;
}
.inspector-card h3 {
  font-size: 13px;
  color: #94a3b8;
  margin-bottom: 8px;
  text-transform: uppercase;
}
.inspector-card pre {
  overflow-x: auto;
}
.inspector-card code {
  font-family: 'Consolas', 'Courier New', monospace;
  font-size: 13px;
  color: #38bdf8;
}
/* Controls */
.control-panel {
  background: #ffffff;
  border-radius: 10px;
```

```

padding: 18px 20px;
border: 1px solid #e2e8f0;
margin-bottom: 16px;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.04);
}
.control-panel h3 {
  font-size: 16px;
  color: #1e293b;
  margin-bottom: 4px;
}
.panel-desc {
  font-size: 12px;
  color: #64748b;
  margin-bottom: 12px;
}
.btn-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 8px;
}
.ctrl-btn {
  background: #f1f5f9;
  border: 1px solid #cbd5e1;
  color: #334155;
  padding: 8px 12px;
  border-radius: 6px;
  font-size: 12px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.2s;
  text-align: center;
}
.ctrl-btn:hover {
  background: #e2e8f0;
  color: #0f172a;
}
.ctrl-btn-reset {
  background: #fee2e2;
  border-color: #fca5a5;
  color: #991b1b;
}
.ctrl-btn-reset:hover {
  background: #fecaca;
}

```

script.js

```

/** Document Object Model (DOM) Manipulation Script
 * Demonstrates: Content, Style, and Attribute Modifications */
document.addEventListener('DOMContentLoaded', () => {

```

```

// 1. Target DOM Element References
const targetCard = document.getElementById('targetCard');
const cardBadge = document.getElementById('cardBadge');
const cardImage = document.getElementById('cardImage');
const cardTitle = document.getElementById('cardTitle');
const cardDescription = document.getElementById('cardDescription');
const featureList = document.getElementById('featureList');
const targetBtn = document.getElementById('targetBtn');
const targetLink = document.getElementById('targetLink');
const codeOutput = document.getElementById('codeOutput');
// Helper to log executed commands in inspector
function logCommand(cmd) {
  codeOutput.textContent += cmd;
}
}
// 1. CONTENT MANIPULATION
// A. Modify text content using .textContent
document.getElementById('btnTextContent').addEventListener('click', () => {
  cardTitle.textContent = "Title Updated via textContent (" + new
Date().toLocaleTimeString() + ")";
  logCommand(`cardTitle.textContent = "Title Updated via textContent (...)";`);
});
// B. Inject formatted HTML markup using .innerHTML
document.getElementById('btnInnerHTML').addEventListener('click', () => {
  cardBadge.innerHTML = '<span style="background: #10b981; color: white; padding:
3px 8px; border-radius: 4px;">✓ Status: Active (Injected via innerHTML)</span>';
  logCommand(`cardBadge.innerHTML = '<span style="background: #10b981; color:
white; ...">✓ Status: Active</span>`;);
});
// C. Append child element to DOM tree
let nodeCounter = 3;
document.getElementById('btnAddNode').addEventListener('click', () => {
  const newItem = document.createElement('li');
  newItem.textContent = `Dynamic DOM Node Item ${nodeCounter++} (Created via
createElement)`;
  newItem.style.color = '#2563eb';
  featureList.appendChild(newItem);
  logCommand(`const li = document.createElement('li');\nli.textContent = "New
Node...";\nfeatureList.appendChild(li);`);
});
// D. Reset Content
document.getElementById('btnResetContent').addEventListener('click', () => {
  cardTitle.textContent = "Default Component Title";
  cardBadge.innerHTML = "Status: Normal";
  cardDescription.textContent = "This is the original paragraph content. JavaScript can
dynamically alter its text, inline styles, classes, and attributes.";
  featureList.innerHTML = '<li>Standard DOM Node Item 1</li><li>Standard DOM
Node Item 2</li>';
  nodeCounter = 3;
  logCommand(`// Content reset to initial default markup`);
});

```

```

// 2. STYLE MANIPULATION
// A. Direct Inline Style Modification (element.style.property)
let isColorToggled = false;
document.getElementById('btnChangeColor').addEventListener('click', () => {
  isColorToggled = !isColorToggled;
  cardDescription.style.color = isColorToggled ? '#d97706' : '#475569';
  cardDescription.style.fontWeight = isColorToggled ? 'bold' : 'normal';
  logCommand(`cardDescription.style.color =
"${cardDescription.style.color}";\ncardDescription.style.fontWeight =
"${cardDescription.style.fontWeight}";`);
});
// B. Toggle CSS Class (classList.toggle)
document.getElementById('btnToggleTheme').addEventListener('click', () => {
  targetCard.classList.toggle('dark-theme');
  const hasDark = targetCard.classList.contains('dark-theme');
  logCommand(`targetCard.classList.toggle('dark-theme'); // isDark: ${hasDark}`);
});
// C. Toggle Highlight Border
document.getElementById('btnToggleBorder').addEventListener('click', () => {
  targetCard.classList.toggle('highlight-border');
  const hasHighlight = targetCard.classList.contains('highlight-border');
  logCommand(`targetCard.classList.toggle('highlight-border'); // Active:
${hasHighlight}`);
});
// D. Increase Font Size dynamically
let currentFontSize = 14;
document.getElementById('btnIncreaseFont').addEventListener('click', () => {
  currentFontSize = currentFontSize >= 20 ? 14 : currentFontSize + 2;
  cardDescription.style.fontSize = currentFontSize + 'px';
  logCommand(`cardDescription.style.fontSize = "${currentFontSize}px";`);
});
// 3. ATTRIBUTE MANIPULATION
// A. Swap image source via setAttribute('src', ...)
let isSecondImage = false;
document.getElementById('btnSwapImage').addEventListener('click', () => {
  isSecondImage = !isSecondImage;
  const newSrc = isSecondImage
    ? "https://images.unsplash.com/photo-1498050108023-c5249f4df085?w=300"
    : "https://images.unsplash.com/photo-1517694712202-14dd9538aa97?w=300";
  const newAlt = isSecondImage ? "Coding on Laptop" : "Laptop Workspace";
  cardImage.setAttribute('src', newSrc);
  cardImage.setAttribute('alt', newAlt);
  logCommand(`cardImage.setAttribute('src', "${newSrc.substring(0,
45)}...");\ncardImage.setAttribute('alt', "${newAlt}");`);
});
// B. Toggle boolean attribute (disabled)
document.getElementById('btnToggleDisabled').addEventListener('click', () => {
  targetBtn.disabled = !targetBtn.disabled;
  targetBtn.textContent = targetBtn.disabled ? "Button Disabled" : "Target Action
Button";

```

```

    logCommand(`targetBtn.disabled = ${targetBtn.disabled};`);
  });
  // C. Modify hyperlink target via setAttribute('href', ...)
  let isGoogleLink = false;
  document.getElementById('btnChangeLink').addEventListener('click', () => {
    isGoogleLink = !isGoogleLink;
    const newUrl = isGoogleLink ? "https://www.google.com" :
"https://developer.mozilla.org";
    const newText = isGoogleLink ? "Visit Google Search &rarr;" : "Visit MDN Web Docs
&rarr;";
    targetLink.setAttribute('href', newUrl);
    targetLink.innerHTML = newText;
    logCommand(`targetLink.setAttribute('href', "${newUrl}");`);
  });
  // D. Update Tooltip attribute (title)
  document.getElementById('btnSetTooltip').addEventListener('click', () => {
    const newTooltip = `Updated Tooltip at ${new Date().toLocaleTimeString()}`;
    targetCard.setAttribute('title', newTooltip);
    logCommand(`targetCard.setAttribute('title', "${newTooltip}");`);
    alert(`Tooltip updated! Hover your mouse over the card to view:\n"${newTooltip}"`);
  });
});

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById('targetCard')</code>	Retrieves a direct DOM element reference by its unique ID.
<code>element.textContent = "..."</code>	Updates the text node directly without HTML parsing, preventing XSS vulnerabilities.
<code>element.innerHTML = "..."</code>	Parses the assigned string as HTML markup and injects child nodes into the element.
<code>document.createElement('li')</code>	Creates a new detached DOM element node of type <code></code> in memory.
<code>element.appendChild(newNode)</code>	Appends a new child node to the end of an element's child node list.
<code>element.style.color = '#d97706'</code>	Mutates the inline CSS color property directly on the element.
<code>element.style.fontSize = "18px"</code>	Sets inline CSS font-size dynamically using camelCase naming.
<code>element.classList.toggle('dark-theme')</code>	Adds the class if not present, or removes it if present.
<code>element.classList.contains('class')</code>	Checks whether the element currently possesses the specified CSS class.
<code>element.setAttribute('src', url)</code>	Sets the value of the specified attribute on the element.
<code>element.setAttribute('href', url)</code>	Changes the target destination of an <code><a></code> anchor tag dynamically.

element.disabled = true/false	Sets the boolean property disabled on form controls.
element.setAttribute('title', text)	Updates the native browser hover tooltip attribute.

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after testing the interactive DOM manipulation controls in a browser.

Test Case	TC-01
Test / Input	Click "Change textContent" and "Inject HTML Badge" buttons
Expected Result	Card title updates text via textContent and green badge is inserted via innerHTML.
Actual Result	Title and badge updated instantly without page refresh; executed commands logged.
Status	Pass

TC-01 Output:

Title Updated via textContent (11:18:05)

This is the original paragraph content. JavaScript can dynamically alter its text, inline styles, classes, and attributes.

- Standard DOM Node Item 1
- Standard DOM Node Item 2

Test Case	TC-02
Test / Input	Click "Toggle Dark Theme" and "Toggle Highlight Border" buttons
Expected Result	Card background transitions to dark #0f172a, text to white, and border turns cyan.
Actual Result	CSS classes toggled dynamically with instant visual theme inversion.
Status	Pass

TC-02 Output:

Document Object Model (DOM) Manipulation

Demonstrating Dynamic Content, Style, and Attribute Updates via JavaScript

Target HTML Element

Status: Normal

Title Updated via textContent (11:18:05)

This is the original paragraph content. JavaScript can dynamically alter its text, inline styles, classes, and attributes.

- Standard DOM Node Item 1
- Standard DOM Node Item 2

Target Action Button

Visit MDN Web Docs →

EXECUTED JAVASCRIPT COMMAND

```
targetCard.classList.toggle('dark-theme'); // isDark: true
```

1. Content Manipulation

Modify innerHTML, textContent, or append new child nodes.

Change textContent

Inject HTML Badge

Append New Node

Reset Content

2. Style Manipulation

Change element.style properties or toggle CSS classes.

Change Text Color

Toggle Dark Theme

Toggle Highlight Border

Increase Font Size

3. Attribute Manipulation

Set or remove attributes (src, href, title, disabled).

Swap Image (src)

Toggle Button disabled

Change Link (href)

Update Tooltip (title)

Document Object Model (DOM) Manipulation

Demonstrating Dynamic Content, Style, and Attribute Updates via JavaScript

Target HTML Element

Status: Normal



Title Updated via textContent (11:18:05)

This is the original paragraph content. JavaScript can dynamically alter its text, inline styles, classes, and attributes.

- Standard DOM Node Item 1
- Standard DOM Node Item 2

Target Action Button

Visit MDN Web Docs →

EXECUTED JAVASCRIPT COMMAND

targetCard.classList.toggle('dark-theme'); // isDark: false

1. Content Manipulation

Modify innerHTML, textContent, or append new child nodes.

Change textContent

Inject HTML Badge

Append New Node

Reset Content

2. Style Manipulation

Change element.style properties or toggle CSS classes.

Change Text Color

Toggle Dark Theme

Toggle Highlight Border

Increase Font Size

3. Attribute Manipulation

Set or remove attributes (src, href, title, disabled).

Swap Image (src)

Toggle Button disabled

Change Link (href)

Update Tooltip (title)

Test Case	TC-03
Test / Input	Click "Toggle Button disabled" and "Change Link (href)" buttons
Expected Result	Button becomes disabled (greyed out), text says "Button Disabled", and link changes destination.
Actual Result	Attributes updated in the DOM tree; button interaction successfully blocked.
Status	Pass

TC-03 Output:

Document Object Model (DOM) Manipulation

Demonstrating Dynamic Content, Style, and Attribute Updates via JavaScript

Target HTML Element

Status: Normal



Title Updated via textContent (11:18:05)

This is the original paragraph content. JavaScript can dynamically alter its text, inline styles, classes, and attributes.

- Standard DOM Node Item 1
- Standard DOM Node Item 2

Button Disabled

Visit MDN Web Docs →

EXECUTED JAVASCRIPT COMMAND

targetBtn.disabled = true;

1. Content Manipulation

Modify innerHTML, textContent, or append new child nodes.

Change textContent

Inject HTML Badge

Append New Node

Reset Content

2. Style Manipulation

Change element.style properties or toggle CSS classes.

Change Text Color

Toggle Dark Theme

Toggle Highlight Border

Increase Font Size

3. Attribute Manipulation

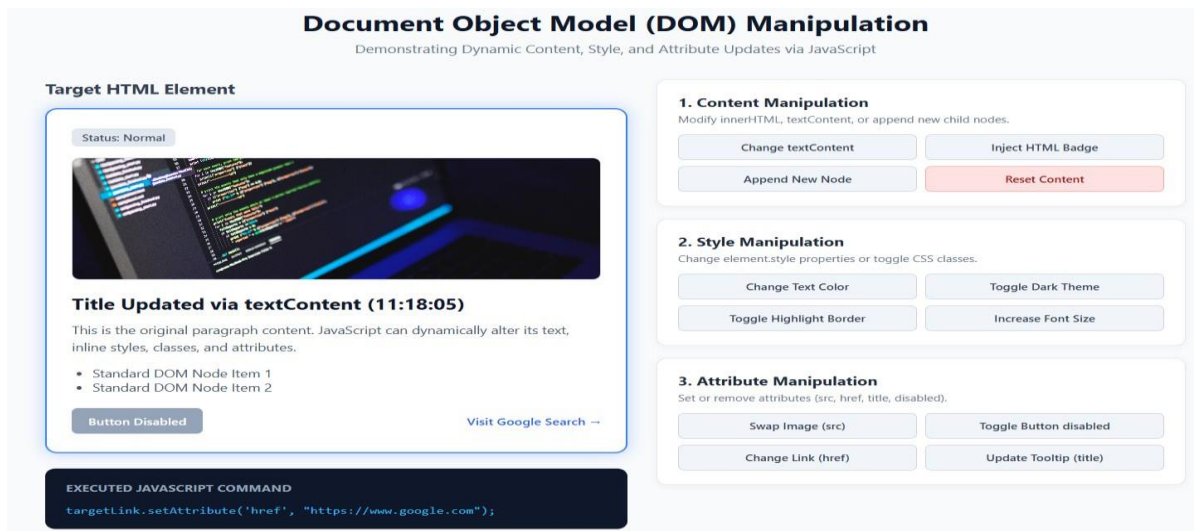
Set or remove attributes (src, href, title, disabled).

Swap Image (src)

Toggle Button disabled

Change Link (href)

Update Tooltip (title)



9. OUTPUT

Output 1: DOM Dynamic Manipulation Sandbox Webpage Output

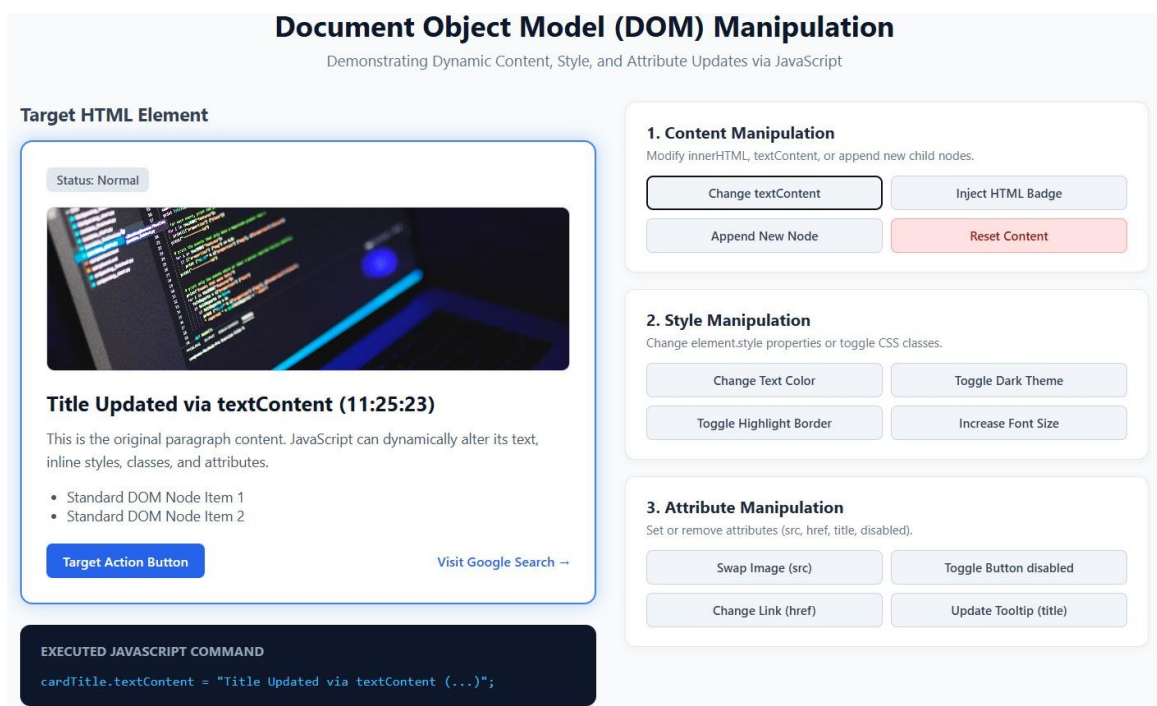


Figure 1: Complete Webpage Output of DOM Manipulation Sandbox

10. RESULT

Thus, the architecture and mechanics of the **Document Object Model (DOM)** were thoroughly examined, and an interactive web application demonstrating dynamic manipulation of content (textContent, innerHTML), styles (style, classList), and attributes (setAttribute, disabled) was successfully developed and verified.